



Blind Signatures From Oblivious Transfer : Analyse et Implémentation

Julian Mouthon

Master 1 Cryptologie, Calcul Haute-Performance et Algorithmes

Stage réalisé au sein de l'équipe Almasty
LIP6, Sorbonne Université, CNRS, Paris, France

Encadrant : Ky Nguyen
22 juin 2026 — 22 août 2026

Résumé

Les signatures aveugles (Blind Signatures) introduites par Chaum en 1982, permettent à un utilisateur d'obtenir une signature sur un message déterminé sans que le signataire n'apprenne son contenu. Ces mécanismes peuvent être utilisés en pratique dans le cadre des votes électroniques, des transactions bancaires ou des protocoles d'identification. Dans ce rapport, nous étudions la construction de BSFOT (Blind Signatures From Oblivious Transfer), basée essentiellement sur les groupes bilinéaires. Nous nous intéressons plus particulièrement à la construction d'un protocole de signature aveugle reposant sur un mécanisme de dual-OT. Nous présentons d'abord les notions et objets mathématiques nécessaires ainsi que le fonctionnement du protocole. Nous analysons ensuite sa validité et ses propriétés de sécurité. Enfin, nous implementons le schéma de signature et mesurons ses performances pour différentes courbes elliptiques et tailles de message.

Table des matières

1	Introduction	2
2	Préliminaires	2
2.1	Notations	2
2.2	Groupes bilinéaires	2
2.3	Signatures aveugles	2
2.4	Transfert Inconscient (OT)	3
2.5	Transfert inconscient dual (dual-OT)	3
2.6	Signature de Waters	3
3	Le protocole dual-OT	4
3.1	Motivation : du mode OT classique au mode dual-OT	4
3.2	Paramètres publics	6
3.3	Schéma de signature	6
3.4	Validité de la signature	7
3.5	Les branches DH et messy	8
4	Preuves de sécurité	8
4.1	Lemmes	8
4.2	Inforgéabilité	10
4.3	Aveuglement	10
5	Implémentation et évaluation expérimentale	12
5.1	Objectifs	12
5.2	Environnement	12
5.3	Implémentation du protocole	12
5.3.1	Architecture	12
5.3.2	Courbes elliptiques utilisées	12
5.4	Résultats expérimentaux	14
5.5	Analyse des résultats	14
A	Benchmarks détaillés	15
A.1	Génération des clés	15
A.2	Exécution côté utilisateur	15
A.3	Exécution côté signataire	16
A.4	Dérivation de la signature	16
A.5	Vérification	17

1 Introduction

Les signatures aveugles (Blind Signatures) introduites par Chaum en 1982, permettent à un utilisateur d'obtenir une signature sur un message déterminé sans que le signataire n'apprenne son contenu. Ces mécanismes peuvent être utilisés en pratique dans le cadre des votes électroniques, des transactions bancaires ou des protocoles d'identification.

Ce rapport présente un nouveau schéma de signatures aveugles utilisant le transfert inconscient (Oblivious Transfer, OT) et les signatures de Waters.

2 Préliminaires

Dans cette partie, nous introduisons les notations et concepts de base nécessaires à la compréhension du protocole présenté dans ce rapport. Le protocole repose principalement sur les signatures de Waters, un schéma de signature basé sur les groupes bilinéaires, et le transfert inconscient (OT), plus précisément le OT dual-mode, qui permet de réaliser un transfert de messages de manière sécurisée.

2.1 Notations

- λ désigne le paramètre de sécurité.
- Un algorithme probabiliste en temps polynomial (PPT) est un algorithme dont le temps d'exécution est polynomial en λ .
- Pour un ensemble fini S , la notation $x \xleftarrow{\$} S$ désigne un tirage uniforme de x dans S .
- Pour $\ell \in \mathbb{N}$, on note $[\ell] = \{1, \dots, \ell\}$.
- Une fonction $f(\lambda)$ est dite négligeable, notée $f(\lambda) = \text{negl}(\lambda)$, si elle décroît plus rapidement que l'inverse de tout polynôme en λ .

2.2 Groupes bilinéaires

Les groupes bilinéaires constituent un élément central des constructions étudiées dans ce rapport. Ils interviennent notamment dans la construction et la vérification des signatures de Waters ainsi que dans les différentes étapes du protocole de transfert inconscient présenté par la suite. Nous introduisons donc ici les principales propriétés des groupes bilinéaires qui seront utilisées dans les sections suivantes.

Soient $\mathbb{G}_1$, $\mathbb{G}_2$ et $\mathbb{G}_T$ trois groupes cycliques d'ordre premier p . Un couplage bilinéaire est une application $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ satisfaisant les propriétés suivantes :

- **Bilinéarité** : pour tout $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$ et $a, b \in \mathbb{Z}_p$,

$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}.$$

- **Non-dégénérescence** : si g_1 et g_2 sont des générateurs, alors

$$e(g_1, g_2) \neq 1_{\mathbb{G}_T}.$$

- **Efficacité** : le calcul du couplage est réalisable en temps polynomial.

2.3 Signatures aveugles

Pour rappel, notre protocole vise à permettre à un utilisateur d'obtenir une signature sur un message tout en préservant la confidentialité de celui-ci vis-à-vis du signataire. Un tel algorithme est appelé schéma de signature aveugle, et est défini par les algorithmes suivants :

- $\text{BSKeygen}(1^\lambda)$: génère une clé publique bvk et une clé secrète bsk ;
- $\text{BSParams}(1^\lambda)$: génère les paramètres publics ;
- $\text{BSUser}(\text{bvk}, M)$: exécuté par l'utilisateur, produit un premier message ρ_1 ainsi qu'un état interne st ;
- $\text{BSSigner}(\text{bsk}, \rho_1)$: exécuté par le signataire, produit une réponse ρ_2 ;
- $\text{BSDerive}(st, \rho_2)$: permet à l'utilisateur d'obtenir une signature σ ;
- $\text{BSVerify}(\text{bvk}, M, \sigma)$: vérifie la validité de la signature.

Une signature est dite valide si $\text{BSVerify}(\text{bvk}, M, \sigma) = 1$.

2.4 Transfert Inconscient (OT)

Une des notions fondamentales de notre construction est le transfert inconscient, que l'on nommera OT (Oblivious Transfer). C'est un protocole permettant à un destinataire d'obtenir une information parmi plusieurs informations proposées par un émetteur, sans que ce dernier ne puisse déterminer le choix effectué, et tel que le destinataire n'apprenne aucune information sur les données qu'il n'a pas sélectionnées

Definition 2.1 (Transfert inconscient). Un protocole de transfert inconscient (1 parmi 2) est un protocole interactif entre un émetteur, en entrée (m_0, m_1) , et un récepteur, en entrée $b \in \{0, 1\}$, tel qu'à l'issue de l'exécution :

- le récepteur apprend m_b et n'obtient aucune information sur m_{1-b} ;
- l'émetteur n'apprend aucune information sur le bit b .

Dans notre cas, il est exécuté indépendamment pour chaque position $i \in [\ell]$ du message $M = (M_1, \dots, M_\ell)$ que l'utilisateur souhaite faire signer. Le signataire joue le rôle de l'émetteur, avec en entrée une paire de valeurs construites à partir d'un partage aléatoire de la clé secrète bsk et de l'aléa t utilisé pour signer. L'utilisateur joue le rôle du récepteur, avec en entrée le bit M_i . À l'issue des ℓ exécutions, l'utilisateur récupère exactement les valeurs nécessaires à la reconstruction d'une signature sur M , sans jamais révéler M au signataire, et sans obtenir d'information sur les valeurs associées aux bits.

2.5 Transfert inconscient dual (dual-OT)

Notre protocole utilise plus précisément une variante du transfert inconscient, le dual-OT (dual Oblivious Transfer), dans lequel chaque exécution repose sur deux branches, l'une associée au bit 0 et l'autre au bit 1, construites de sorte qu'une seule d'entre elles soit exploitable.

Definition 2.2 (Dual-OT). Un protocole dual-OT est un protocole interactif dans lequel, pour chaque choix $b \in \{0, 1\}$ du récepteur, les deux branches associées à ce choix sont constituées de telle sorte que :

- la branche correspondant au choix b est une paire Diffie-Hellman (DH) ;
- la branche correspondant au choix $1 - b$ est une paire messy, c'est-à-dire une paire qui ne satisfait pas la relation Diffie-Hellman.

Cette structure à deux branches, détaillée plus tard à la section 3.5, est ce qui distingue le dual-OT du OT classique, un utilisateur malhonnête ne peut exploiter qu'une seule branche par position.

2.6 Signature de Waters

La construction utilise la fonction de hachage de Waters permettant d'encoder un message binaire dans un élément du groupe $\mathbb{G}_1$. Soit un message $M = (M_1, \dots, M_\ell) \in \{0, 1\}^\ell$. Les paramètres publics de

la fonction sont constitués de $u_0, u_1, \dots, u_\ell \in \mathbb{G}_1$. La valeur associée au message est définie par :

$$F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i}.$$

Ainsi, chaque bit M_i influence la valeur de hachage par l'intermédiaire du facteur $u_i^{M_i}$.

En partant de cette fonction de hachage, il est possible d'arriver à un mécanisme de signature, c'est celui-ci que nous utilisons. Le schéma de signature de Waters utilise les paramètres publics précédents ainsi qu'une clé secrète $a \in \mathbb{Z}_p$. La clé publique est $\text{bvk} = g_2^a$. Pour signer un message M , le signataire choisit un aléa $t \xleftarrow{\$} \mathbb{Z}_p$ et calcule :

$$\sigma_1 = h_s^a F(M)^t, \quad \sigma_2 = g_2^t.$$

La signature est alors

$$\sigma = (F(M), \sigma_1, \sigma_2).$$

La vérification consiste à vérifier l'équation de couplage

$$e(h_s, \text{bvk}) \cdot e(F(M), \sigma_2) \stackrel{?}{=} e(\sigma_1, g_2).$$

3 Le protocole dual-OT

3.1 Motivation : du mode OT classique au mode dual-OT

Notre étude portait originellement sur un schéma de signature aveugle basé sur le mode OT classique, qui, comme le mode dual-OT, permet de réaliser un transfert de messages de manière sécurisée. Le protocole OT classique n'est pas complet du point de vue des propriétés de sécurité, puisqu'il ne dispose pas d'une preuve d'inforgeabilité, contrairement au protocole dual-OT. Il a servi de base et est utilisé dans notre implémentation (voir section 5.3) comme comparateur pour le protocole dual-OT. Il est présenté ci-dessous comme référence. C'est pourquoi nous avons choisi de nous concentrer sur le dual-OT pour la suite de notre étude.

Paramètres publics

- un système de groupes bilinéaires $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, où $\mathbb{G}_1$ et $\mathbb{G}_2$ sont des groupes cycliques d'ordre premier p , munis d'un couplage bilinéaire $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, avec des générateurs $g_1 \in \mathbb{G}_1$ et $g_2 \in \mathbb{G}_2$.
- les paramètres de la fonction de hachage de Waters, constitués d'un élément aléatoire $u_0 \in \mathbb{G}_1$ ainsi que d'un vecteur $\mathbf{u} = (u_1, \dots, u_\ell) \in \mathbb{G}_1^\ell$.
- un vecteur $\text{ek} = (h_i)_{i=1}^\ell$ avec $h_i = g_1^{x_i}$ obtenu à partir de scalaires aléatoires $x_i \xleftarrow{\$} \mathbb{Z}_p$.
- la clé publique du schéma de signature $\text{bvk} = g_2^a$, associée à la clé secrète $\text{bsk} = h_s^a$, avec $h_s = \prod_{i=1}^\ell h_i$.

Schéma de signature

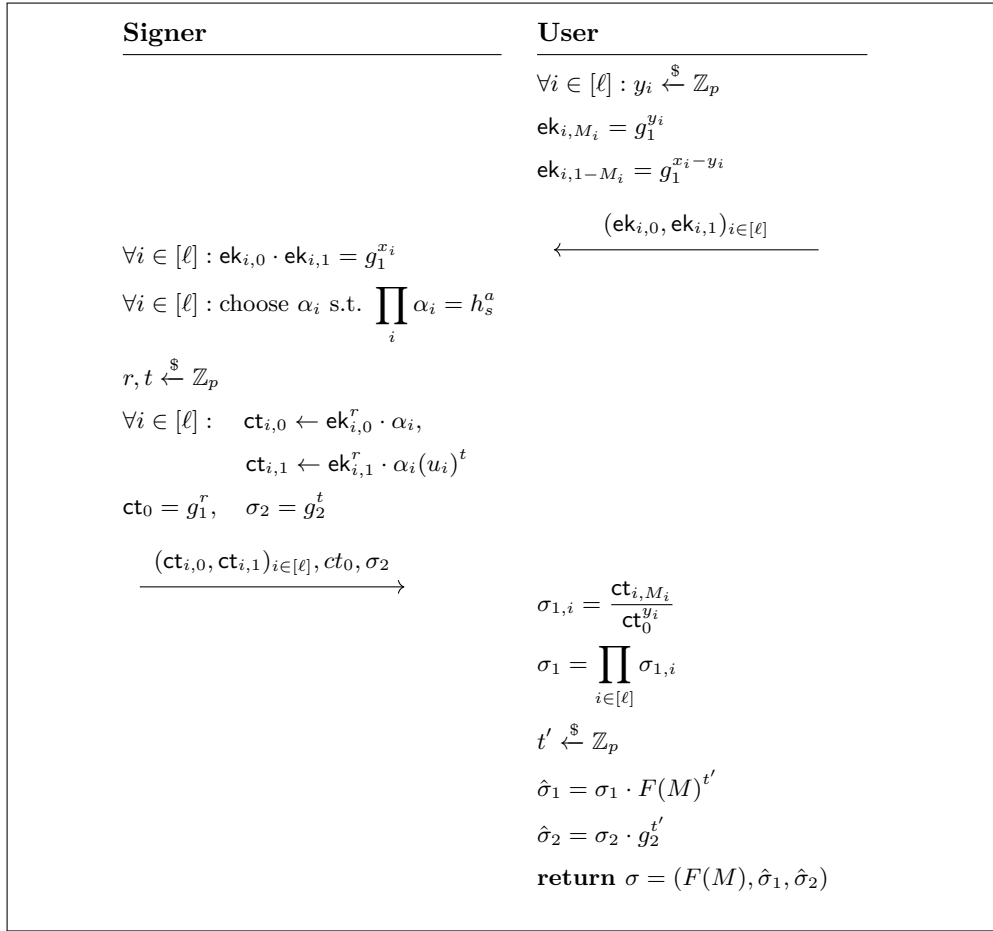


FIGURE 1 – Schéma de signature aveugle basé sur le OT classique

Validité de la signature

La signature obtenue par l'utilisateur est une signature de Waters de la forme $\sigma = (F(M), \hat{\sigma}_1, \hat{\sigma}_2)$ avec :

$$\hat{\sigma}_1 = \sigma_1 \cdot F(M)^{t'} = h_s^a \cdot F(M)^{t+t'}, \quad \hat{\sigma}_2 = \sigma_2 \cdot g_2^{t'} = g_2^{t+t'}.$$

On remarque que le signataire ne connaît pas la valeur $t + t'$, car le terme t' est choisi aléatoirement par l'utilisateur après la phase de signature. La vérification s'effectue ensuite à l'aide de la clé publique du signataire $\mathbf{bvk} = g_2^a$ en vérifiant l'équation de couplage :

$$e(h_s, \mathbf{bvk}) \cdot e(F(M), \hat{\sigma}_2) \stackrel{?}{=} e(\hat{\sigma}_1, g_2).$$

En remplaçant les valeurs obtenues par l'utilisateur :

$$\begin{aligned}
e(h_s, \mathbf{bvk}) \cdot e(F(M), \hat{\sigma}_2) &= e(h_s, g_2^a) \cdot e(F(M), g_2^{t+t'}) \\
&= e(h_s^a, g_2) \cdot e(F(M)^{t+t'}, g_2) \quad \text{par bilinéarité du couplage} \\
&= e(h_s^a \cdot F(M)^{t+t'}, g_2) \quad \text{par bilinéarité et multiplicativité} \\
&= e(\hat{\sigma}_1, g_2)
\end{aligned}$$

Ainsi, l'équation de vérification est satisfaite et la signature obtenue est une signature de Waters valide.

3.2 Paramètres publics

Durant l'exécution du protocole, les deux parties (utilisateur et signataire) disposent de paramètres en commun et publiquement accessibles. Ces paramètres sont les suivants :

- un système de groupes bilinéaires $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, où $\mathbb{G}_1$ et $\mathbb{G}_2$ sont des groupes cycliques d'ordre premier p , munis d'un couplage bilinéaire $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, avec des générateurs $g_1 \in \mathbb{G}_1$ et $g_2 \in \mathbb{G}_2$.
- les paramètres de la fonction de hachage de Waters, constitués d'un élément aléatoire $u_0 \in \mathbb{G}_1$ ainsi que d'un vecteur $\mathbf{u} = (u_1, \dots, u_\ell) \in \mathbb{G}_1^\ell$. La fonction de hachage associée est toujours : $F(M) = u_0 \prod_{i=1}^\ell u_i^{M_i}$.
- un CRS global : OT-CRS = $(g, h, w, \tilde{w})$, où les éléments sont choisis uniformément dans le groupe approprié. On définit également : $\eta = \log_g(h)$, lorsque $g \neq 1$.
- la clé publique du schéma de signature : $\text{bvk} = g_2^a$, associée à la clé secrète : $\text{bsk} = h_s^a$, où $h_s \in \mathbb{G}_1$ est choisi aléatoirement.

3.3 Schéma de signature

Le protocole de signature se déroule comme ci-dessous, avec l'utilisateur à droite qui initie la communication et le signataire à gauche qui répond. Les messages échangés entre les deux parties sont représentés par des flèches horizontales. A la fin du protocole, l'utilisateur obtient une signature sur M , qui sera vérifiée après par le vérifieur (voir section 3.4).

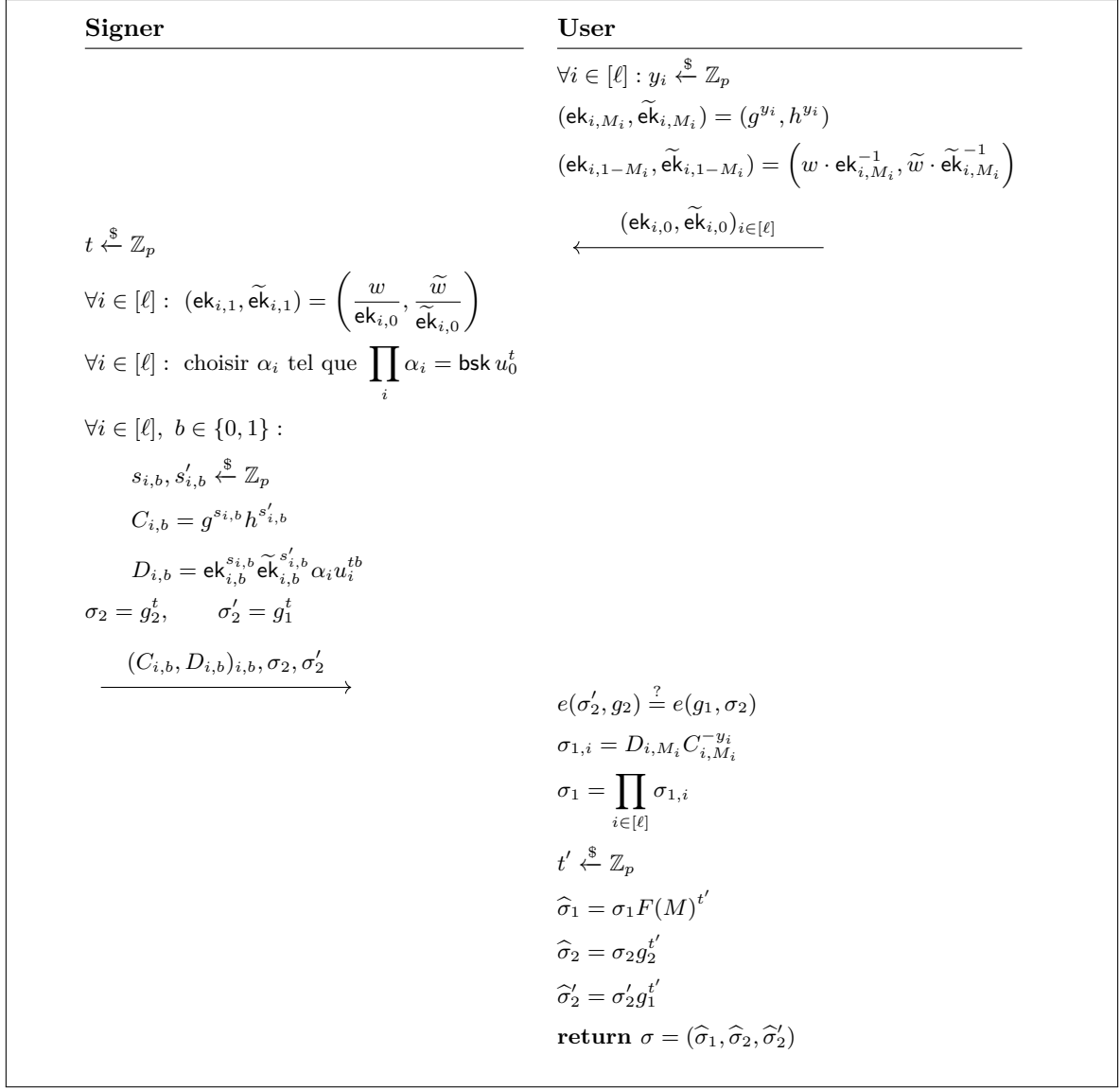


FIGURE 2 – Schéma de signature aveugle basé sur le dual-OT

3.4 Validité de la signature

Après avoir obtenu la signature, l'utilisateur peut la transmettre à un vérifieur qui testera sa validité de la manière suivante :

$$\text{Signature acceptée} \iff \begin{cases} e(\widehat{\sigma}'_2, g_2) = e(g_1, \widehat{\sigma}_2), \\ e(\widehat{\sigma}_1, g_2) = e(h_s, \mathbf{bvk}) \cdot e(F(M), \widehat{\sigma}_2). \end{cases}$$

Première équation (en utilisant la bilinéarité) :

$$\begin{aligned} e(\widehat{\sigma}'_2, g_2) &= e(g_1^{t+t'}, g_2) \\ &= e(g_1, g_2)^{t+t'} \\ &= e(g_1, g_2^{t+t'}) \\ &= e(g_1, \widehat{\sigma}_2). \end{aligned}$$

Seconde équation (toujours par bilinéarité) :

$$\begin{aligned}
e(\widehat{\sigma}_1, g_2) &= e(h_s^a F(M)^{t+t'}, g_2) \\
&= e(h_s^a, g_2) e(F(M)^{t+t'}, g_2) \\
&= e(h_s, g_2^a) e(F(M), g_2^{t+t'}) \\
&= e(h_s, \text{bvk}) e(F(M), \widehat{\sigma}_2),
\end{aligned}$$

3.5 Les branches DH et messy

Durant l'exécution du protocole, plus précisément lors de l'étape **BSUser** (voir section 2.3), l'utilisateur construit deux branches pour chaque position i du message M . La première correspond à son vrai bit M_i et est définie par

$$(\text{ek}_{i,M_i}, \widetilde{\text{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i}),$$

où y_i est choisi aléatoirement. Comme $h = g^\eta$, on obtient $h^{y_i} = (g^{y_i})^\eta$, c'est-à-dire $\widetilde{\text{ek}}_{i,M_i} = \text{ek}_{i,M_i}^\eta$. Cette branche vérifie donc la relation de Diffie-Hellman et est appelée *branche DH*. C'est la branche sur laquelle le protocole fonctionne normalement. La seconde branche est construite à partir de la première :

$$(\text{ek}_{i,1-M_i}, \widetilde{\text{ek}}_{i,1-M_i}) = \left(\frac{w}{\text{ek}_{i,M_i}}, \frac{\widetilde{w}}{\widetilde{\text{ek}}_{i,M_i}} \right).$$

Pour que cette branche soit également une paire DH, il faudrait que

$$\widetilde{\text{ek}}_{i,1-M_i} = \text{ek}_{i,1-M_i}^\eta.$$

Or,

$$\text{ek}_{i,1-M_i}^\eta = \left(\frac{w}{\text{ek}_{i,M_i}} \right)^\eta = \frac{w^\eta}{\widetilde{\text{ek}}_{i,M_i}}.$$

Cette égalité serait satisfaite uniquement si $\widetilde{w} = w^\eta$. Cependant, le CRS est choisi de manière à vérifier $\widetilde{w} \neq w^\eta$. La seconde branche ne peut donc jamais être une paire DH. C'est une *branche messy*.

En résumé, pour un utilisateur honnête, chaque position contient exactement une branche DH (la bonne branche, correspondant au bit du message) et une branche messy (la mauvaise branche). Elle ne transporte aucune information utile.

4 Preuves de sécurité

4.1 Lemmes

Lemma 4.1 (Uniformité des branches messy (non-DH)). *Supposons que $g \neq 1$ et que $\eta = \log_g h$. Soit $(u, v) \in \mathbb{G}_1^2$ une paire non-DH, c'est-à-dire telle que $v \neq u^\eta$. Alors la variable aléatoire*

$$(C, \text{mask}) := (g^s h^{s'}, u^s v^{s'}), \quad (s, s') \xleftarrow{\$} \mathbb{Z}_p^2,$$

est uniformément distribuée sur $\mathbb{G}_1^2$. Par conséquent, pour tout message clair $m \in \mathbb{G}_1$, le chiffré défini par

$$(C, D) = (g^s h^{s'}, u^s v^{s'} \cdot m)$$

est uniformément distribué sur $\mathbb{G}_1^2$, avec une distribution qui ne dépend pas de m .

Démonstration. Puisque g est un générateur de $\mathbb{G}_1$, on peut écrire $u = g^c$ et $v = g^e$ pour des $c, e \in \mathbb{Z}_p$

uniques. L'hypothèse $v \neq u^\eta$ implique $e \neq c\eta \pmod{p}$. Considérons l'application $\mathbb{Z}_p$ -linéaire

$$\varphi : \mathbb{Z}_p^2 \rightarrow \mathbb{Z}_p^2, \quad \varphi(s, s') = (s + \eta s', cs + es'),$$

dont la matrice est

$$\begin{pmatrix} 1 & \eta \\ c & e \end{pmatrix}, \quad \det = e - c\eta.$$

Comme $e - c\eta \neq 0$ dans le corps $\mathbb{Z}_p$, φ est une bijection de $\mathbb{Z}_p^2$. Puisque (s, s') est uniformément distribué sur $\mathbb{Z}_p^2$, $\varphi(s, s')$ l'est également. Enfin, l'application $x \mapsto g^x$ est une bijection de $\mathbb{Z}_p$ vers $\mathbb{G}_1$ puisque g est un générateur d'un groupe d'ordre premier. Appliquée à chaque coordonnée, elle transforme donc $\varphi(s, s')$ en (C, mask) , qui est ainsi uniformément distribué sur $\mathbb{G}_1^2$. Pour la seconde affirmation, $D = \text{mask} \cdot m$. Pour tout m fixé, la multiplication par m est une bijection de $\mathbb{G}_1$. Ainsi, (C, D) est uniformément distribué sur $\mathbb{G}_1^2$; en particulier, sa distribution est la même pour tout m . $\square$

Lemma 4.2 (Randomisation des signatures de Waters). *Soit $\sigma = (\sigma_1, \sigma_2) = (h_s^a F(M)^t, g_2^t)$ une signature de Waters valide, où $t \xleftarrow{\$} \mathbb{Z}_p$. Pour tout $t' \xleftarrow{\$} \mathbb{Z}_p$, la signature $\hat{\sigma} = (\sigma_1 F(M)^{t'}, \sigma_2 g_2^{t'})$ est une signature valide sur le même message M et sa distribution est indépendante de l'aléa initial t .*

Démonstration. On a

$$\begin{aligned} \hat{\sigma}_1 &= \sigma_1 F(M)^{t'} \\ &= h_s^a F(M)^t F(M)^{t'} \\ &= h_s^a F(M)^{t+t'}, \end{aligned}$$

et

$$\hat{\sigma}_2 = \sigma_2 g_2^{t'} = g_2^{t+t'}.$$

Comme $t' \xleftarrow{\$} \mathbb{Z}_p$, la valeur $t + t'$ est uniformément distribuée dans $\mathbb{Z}_p$. La signature randomisée est donc distribuée comme une signature de Waters générée avec un nouvel aléa uniforme, indépendamment de l'aléa initial t . $\square$

Lemma 4.3 (Indistinguabilité sous DDH). *Les distributions obtenues dans le protocole dual-OT en utilisant $F(M)$ ou $F(0)$ sont computationnellement indistinguables.*

Démonstration. On suppose qu'il existe un adversaire $\mathcal{A}$ capable de distinguer les deux distributions. On va construire un distinguéur DDH $\mathcal{B}$, qui a pour but de déterminer si une instance DDH est une instance réelle ou une instance aléatoire. $\mathcal{B}$ reçoit une instance DDH (g_1, g_1^a, g_1^b, c) , où c est soit g_1^{ab} , soit un élément uniforme de $\mathbb{G}_1$. On va utiliser cette instance pour construire les paramètres de la simulation, tels que si $c = g_1^{ab}$, la distribution obtenue corresponde à l'exécution qui utilise $F(M)$, et tels que si c est uniforme, elle corresponde à l'exécution avec $F(0)$.

On veut construire les paramètres de la fonction de Waters $u_0, u_1, \dots, u_\ell$ tels que dans le cas réel, la fonction contienne g_1^{ab} et tels que dans le cas aléatoire elle contienne une valeur indépendante de M . Soit $M \in \{0, 1\}^\ell$ et soit $j \in [\ell]$ tel que $M_j = 1$. Le simulateur choisit $r_0, r_1, \dots, r_\ell \xleftarrow{\$} \mathbb{Z}_p$ et définit les paramètres de Waters par

$$u_0 = g_1^{r_0}, \quad u_j = c g_1^{-r_0}, \quad u_i = g_1^{r_i} \quad \text{pour } i \neq j.$$

avec

$$c = \begin{cases} g_1^{ab} & \text{si l'instance DDH est réelle,} \\ g_1^z, \quad z \xleftarrow{\$} \mathbb{Z}_p & \text{si l'instance DDH est aléatoire,} \end{cases}$$

On a donc, pour la fonction de Waters, avec $M_j = 1$,

$$\begin{aligned}
F(M) &= u_0 \prod_{i=1}^{\ell} u_i^{M_i} \\
&= u_0 u_j \prod_{\substack{i=1 \\ i \neq j}}^{\ell} u_i^{M_i} \\
&= g_1^{r_0} (c g_1^{-r_0}) \prod_{\substack{i=1 \\ i \neq j}}^{\ell} (g_1^{r_i})^{M_i} \\
&= c \prod_{\substack{i=1 \\ i \neq j}}^{\ell} g_1^{r_i M_i}.
\end{aligned}$$

et pour $F(0)$,

$$F(0) = u_0 \prod_{i=1}^{\ell} u_i^0 = u_0 = g_1^{r_0},$$

Par conséquent, lorsque $c = g_1^{ab}$, la valeur $F(M)$ contient le terme g_1^{ab} , tandis que lorsque c est uniformément distribué dans $\mathbb{G}_1$, la valeur $F(M)$ est uniformément distribuée dans $\mathbb{G}_1$ et sa distribution est indépendante du message M .

Ainsi, si l'adversaire $\mathcal{A}$ était capable de distinguer les distributions obtenues avec $F(M)$ et $F(0)$, le distingueur $\mathcal{B}$ pourrait utiliser $\mathcal{A}$ pour déterminer si $c = g_1^{ab}$ ou si c est uniformément distribué dans $\mathbb{G}_1$. Cela permettrait à $\mathcal{B}$ de distinguer une instance DDH réelle d'une instance aléatoire, ce qui contredit l'hypothèse DDH. On en déduit que les distributions obtenues en utilisant $F(M)$ et $F(0)$ sont computationnellement indistinguables. $\square$

4.2 Inforgeabilité

4.3 Aveuglement

Définition 4.4 (Propriété d'aveuglement, signataire honnête). Un schéma de signature aveugle est aveugle pour un signataire honnête si pour tout adversaire PPT $\mathcal{A}$,

$$\left| \Pr \left[\begin{array}{l} (\text{bvk}, \text{bsk}) \leftarrow \text{BSKeygen}(1^\lambda), (m_0, m_1) \leftarrow \mathcal{A}(\text{bvk}), b \xleftarrow{\$} \{0, 1\}, \\ (\rho_{1,i}, st_i) \leftarrow \text{BSUser}(\text{bvk}, m_i) \quad (i \in \{0, 1\}), \\ (\rho_{2,b}, \rho_{2,1-b}) \leftarrow \mathcal{A}(\rho_{1,b}, \rho_{1,1-b}), \\ \sigma_i \leftarrow \text{BSDerive}(st_i, \rho_{2,i}) \quad (i \in \{0, 1\}), \\ \text{si } \exists i, \text{BSVerify}(\text{bvk}, m_i, \sigma_i) = 0 : \sigma_0 = \sigma_1 = \perp, \\ b = \mathcal{A}(\sigma_0, \sigma_1) \end{array} \right] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

Preuve : aveuglement pour un signataire honnête. On veut montrer que, pour deux messages $m_0, m_1 \in \{0, 1\}^\ell$, la vue du signataire est indistinguishable : $\text{View}_{\mathcal{A}}(m_0) \approx \text{View}_{\mathcal{A}}(m_1)$, où $\mathcal{A}$ désigne le signataire honnête. Nous raisonnons par une suite de jeux.

G₀.

Il s'agit de l'exécution réelle du protocole avec le message m_0 . Pour chaque position i , l'utilisateur construit une branche DH correspondant au bit $m_{0,i}$ et une branche messy correspondant à l'autre valeur. Le signataire calcule ensuite les chiffrés des deux branches et les transmet à l'utilisateur.



G₁.

Dans un CRS honnêtement généré, chaque position possède une branche DH et une branche messy. Pour la branche messy, le couple $(\text{ek}_{i,1-M_i}, \tilde{\text{ek}}_{i,1-M_i})$ est une paire non-DH. D'après le lemme 4.1, le chiffrement réalisé sur cette branche est uniformément distribué sur $\mathbb{G}_1^2$, avec une distribution indépendante du message chiffré. On peut donc remplacer, pour chaque position, le chiffré de la branche messy par un couple uniformément distribué dans $\mathbb{G}_1^2$ sans modifier la vue du signataire. Ainsi,

$$G_0 \approx G_1.$$



G₂.

Après le jeu précédent, les chiffrés des branches messy sont uniformes et ne dépendent plus du message. Il reste à traiter les branches DH (non messy). Pour une branche DH correspondant au bit M_i , on a

$$(\text{ek}_{i,M_i}, \tilde{\text{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i}), \quad y_i \xleftarrow{\$} \mathbb{Z}_p.$$

Le signataire observe les deux branches sans savoir laquelle correspond au bit M_i . Sous l'hypothèse DDH, il ne peut pas distinguer la paire (g^{y_i}, h^{y_i}) d'une paire de la forme (g^{y_i}, z_i) , $y_i, z_i \xleftarrow{\$} \mathbb{Z}_p$. On peut donc remplacer la branche DH par une paire indépendante de la branche correspondant au bit du message. Après cela, les deux branches associées à chaque position ont la même distribution quel que soit le bit M_i . Par conséquent, la distribution de l'ensemble du premier message envoyé par l'utilisateur ne dépend plus de M . Ainsi,

$$G_1 \approx G_2$$



G₃.

Après le jeu précédent, les branches envoyées par l'utilisateur sont indépendantes du message M . Après la réponse du signataire, le déchiffrement de la branche DH donne la même forme que dans le protocole classique :

$$\sigma_1 = h_s^a F(M)^t, \quad \sigma_2 = g_2^t, \quad \sigma'_2 = g_1^t.$$

On applique alors la réduction DDH de la section 4.3. Celle-ci permet de remplacer, sous l'hypothèse DDH, le terme $F(M)$ par $F(0)$ dans la signature. Ainsi, la signature simulée ne dépend plus du message M et

$$G_2 \approx G_3.$$



G₄.

Après avoir reçu la réponse du signer, l'utilisateur effectue la phase de déchiffrement. Puis, il effectue la randomisation de Waters en choisissant un aléa $t' \xleftarrow{\$} \mathbb{Z}_p$ et en calculant

$$\hat{\sigma}_1 = \sigma_1 F(M)^{t'}, \quad \hat{\sigma}_2 = \sigma_2 g_2^{t'}, \quad \hat{\sigma}'_2 = \sigma'_2 g_1^{t'}.$$

Comme t' est choisi uniformément dans $\mathbb{Z}_p$, la valeur $t + t'$ est également uniformément distribuée dans $\mathbb{Z}_p$. Ainsi, la distribution de la signature finale est indépendante du message. Elle peut donc être remplacée

par une signature simulée utilisant un aléa indépendant du message. Ainsi,

$$G_3 \approx G_4.$$



G_5 .

Dans ce jeu, le message m_0 est remplacé par m_1 . D’après la transition précédente, la distribution de la signature finale est indépendante du message. Ainsi,

$$G_4 \approx G_5.$$

Conclusion.

On a $G_0 \approx G_1 \approx G_2 \approx G_3 \approx G_4 \approx G_5$, par conséquent, $\text{View}_{\mathcal{A}}(m_0) \approx \text{View}_{\mathcal{A}}(m_1)$, et le protocole satisfait donc la propriété d’aveuglement pour un signataire honnête. □

5 Implémentation et évaluation expérimentale

5.1 Objectifs

Le but d’avoir une implémentation de ces deux protocoles (OT et dual-OT), est de pouvoir évaluer leur performance sur différentes tailles de messages et courbes elliptiques, et de les comparer. Pour avoir des résultats plus précis, il est intéressant de mesurer le temps d’exécution de chaque étape du protocole indépendamment (voir section 2.3, 3.3).

5.2 Environnement

5.3 Implémentation du protocole

5.3.1 Architecture

5.3.2 Courbes elliptiques utilisées

Pour tester le protocole, nous avons utilisé plusieurs courbes elliptiques, plus précisément des courbes dites “pairing-friendly”, c’est-à-dire des courbes permettant de calculer efficacement un couplage bilinéaire. Afin d’obtenir des résultats variés et significatifs, nous avons choisi un large panel de courbes présentant des tailles de corps et des degrés différents. Deux paramètres sont particulièrement importants pour caractériser ces courbes : la taille du corps fini, notée p , et le degré, noté k .

— Taille du corps fini p

La courbe elliptique est définie sur un corps fini $\mathbb{F}_p$, où p est un grand nombre premier. Les éléments manipulés sur la courbe sont donc représentés à l’aide d’éléments de ce corps, dont la taille est d’environ $\log_2(p)$ bits. Ainsi, une courbe ayant une taille de p de 381 bits est définie sur un corps dont les éléments peuvent être représentés sur 381 bits. Cette taille influence notamment le coût des opérations arithmétiques effectuées sur la courbe.

— Degré k

Le degré k représente quant à lui la relation entre le corps $\mathbb{F}_p$ et le sous-groupe d’ordre r utilisé pour le couplage. Il est défini comme le plus petit entier strictement positif vérifiant $p^k \equiv 1 \pmod{r}$. Il indique donc le plus petit degré d’extension du corps $\mathbb{F}_p$ dans lequel les valeurs du couplage peuvent être représentées. En effet, le couplage prend ses valeurs dans le corps étendu $\mathbb{F}_{p^k}$. Par exemple, une

courbe de degré $k = 12$, comme BLS12-381, nécessite de travailler dans une extension $\mathbb{F}_{p^{12}}$ pour effectuer le couplage. Une courbe avec $k = 24$, comme BLS24-315, utilise quant à elle une extension $\mathbb{F}_{p^{24}}$. Plus k est élevé, plus les opérations dans le corps d'extension peuvent être coûteuses.

Les courbes retenues couvrent ainsi plusieurs combinaisons de tailles de corps et de degrés, ce qui permet d'évaluer le comportement du protocole dans des configurations différentes.

Courbe	Degré k	Taille de p
BN254	12	254 bits
BN446	12	446 bits
BLS12-377	12	377 bits
BLS12-381	12	381 bits
BLS12-446	12	446 bits
BLS12-455	12	455 bits
KSS16-330	16	330 bits
KSS18-354	18	354 bits
KSS18-638	18	638 bits
BLS24-315	24	315 bits
BLS48-575	48	575 bits
FM16-765	16	765 bits
FM18-768	18	768 bits

TABLE 1 – Courbes elliptiques utilisées pour les expérimentations.

5.4 Résultats expérimentaux

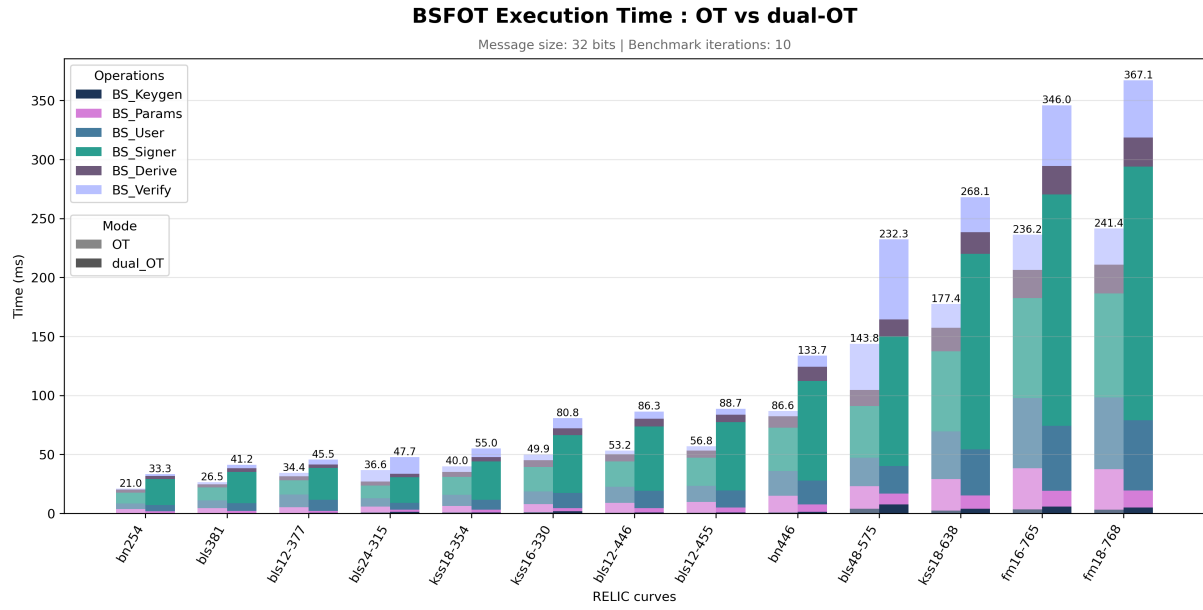


FIGURE 3 – Benchmark global de comparaison entre le protocole OT et le protocole dual-OT pour différentes courbes elliptiques.

5.5 Analyse des résultats

A Benchmarks détaillés

Cette annexe présente les mesures détaillées obtenues pour chacune des étapes des protocoles OT et dual-OT individuellement. Ces graphiques sont présentés dans la section 5.4.

A.1 Génération des clés

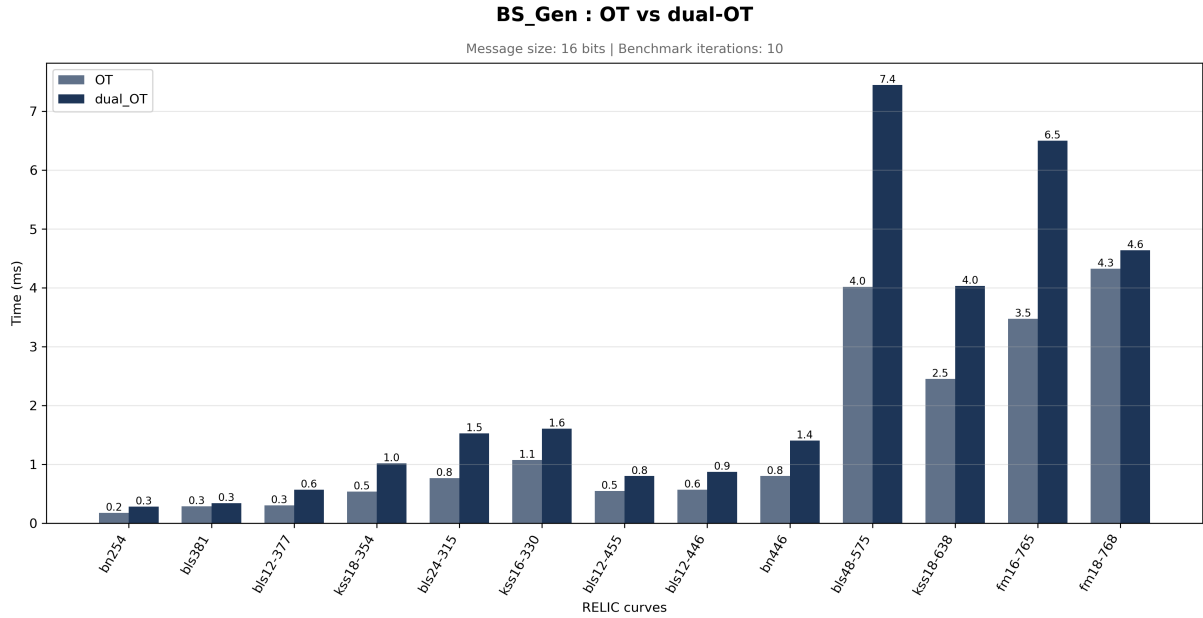


FIGURE 4 – Temps d'exécution de la génération des clés pour les protocoles OT et dual-OT.

A.2 Exécution côté utilisateur

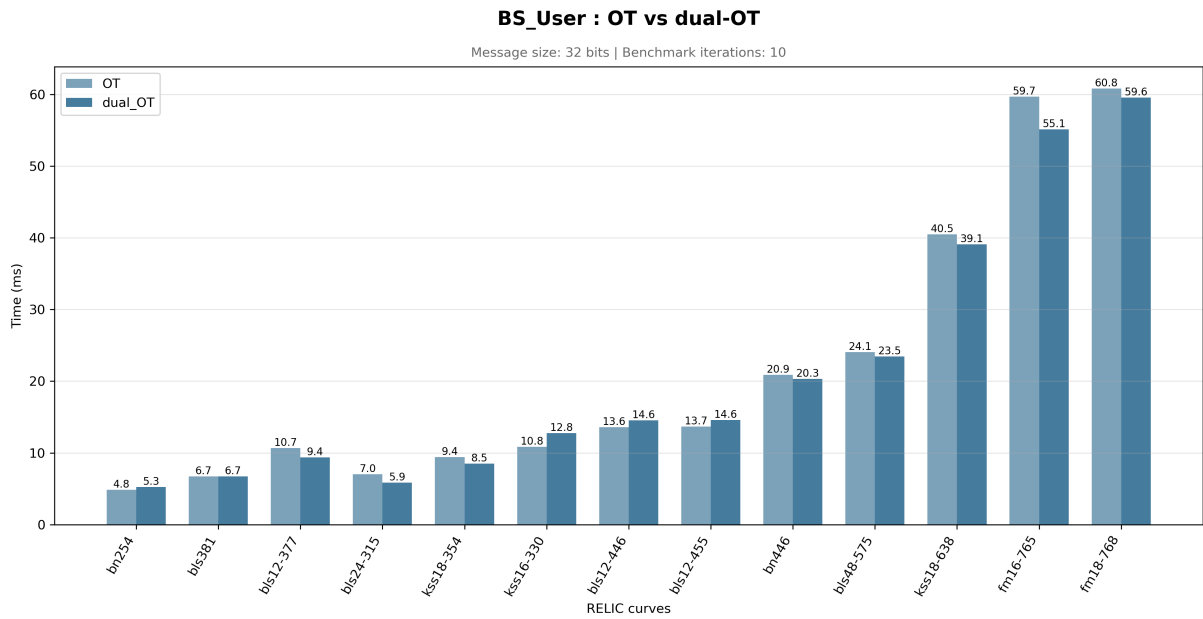


FIGURE 5 – Temps d'exécution de l'étape utilisateur pour les protocoles OT et dual-OT.

A.3 Exécution côté signataire

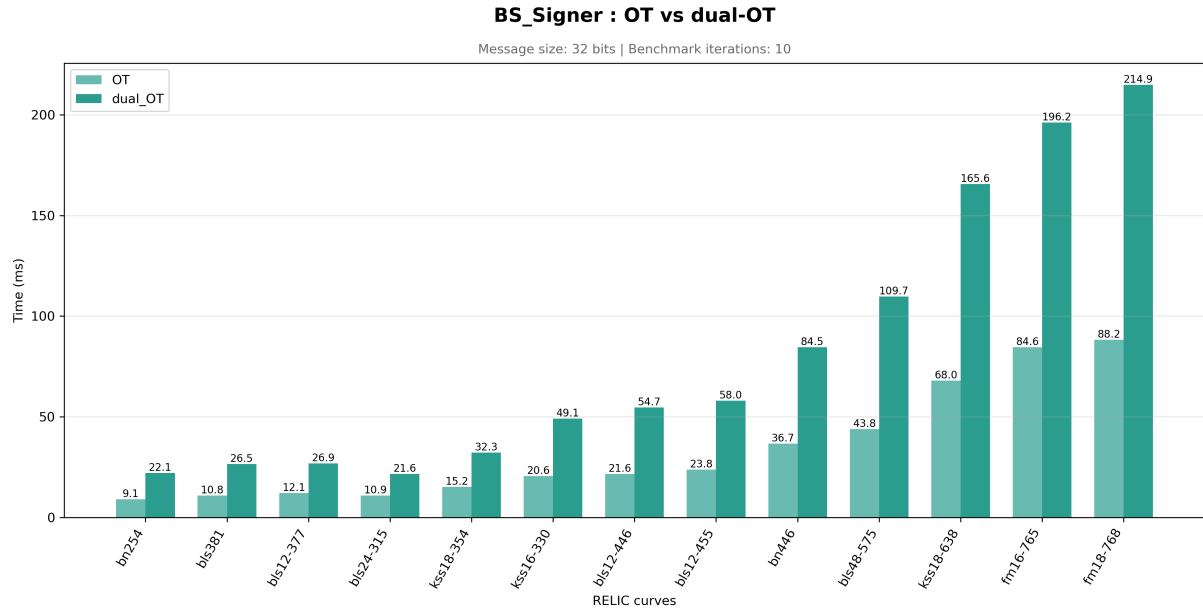


FIGURE 6 – Temps d'exécution de l'étape du signataire pour les protocoles OT et dual-OT.

A.4 Dérivation de la signature

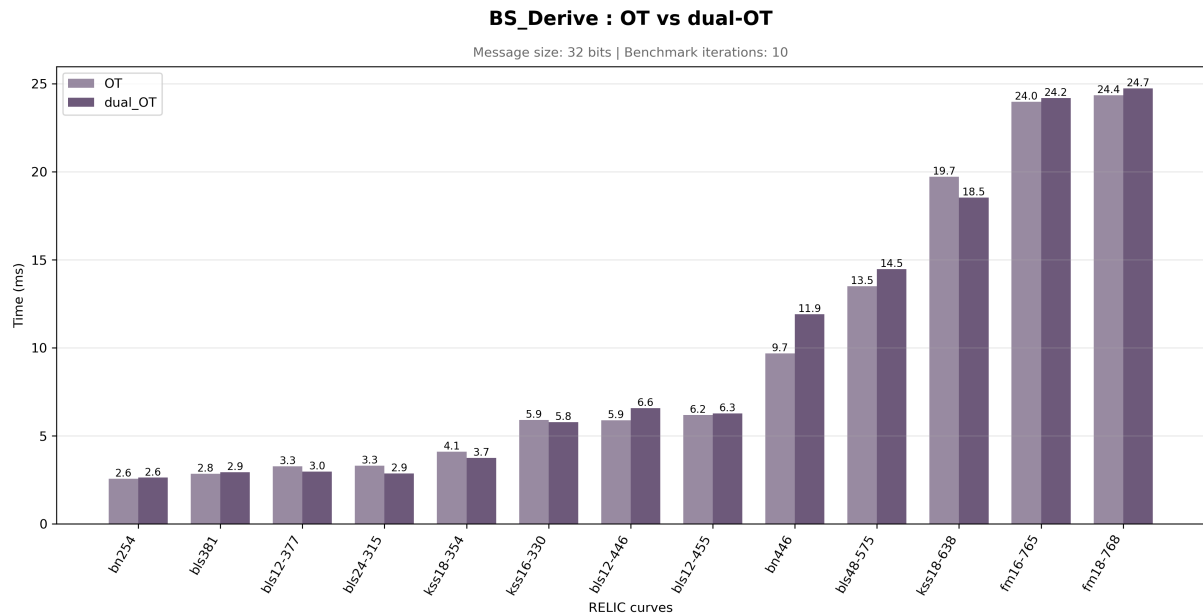


FIGURE 7 – Temps d'exécution de la dérivation de la signature pour les protocoles OT et dual-OT.

A.5 Vérification

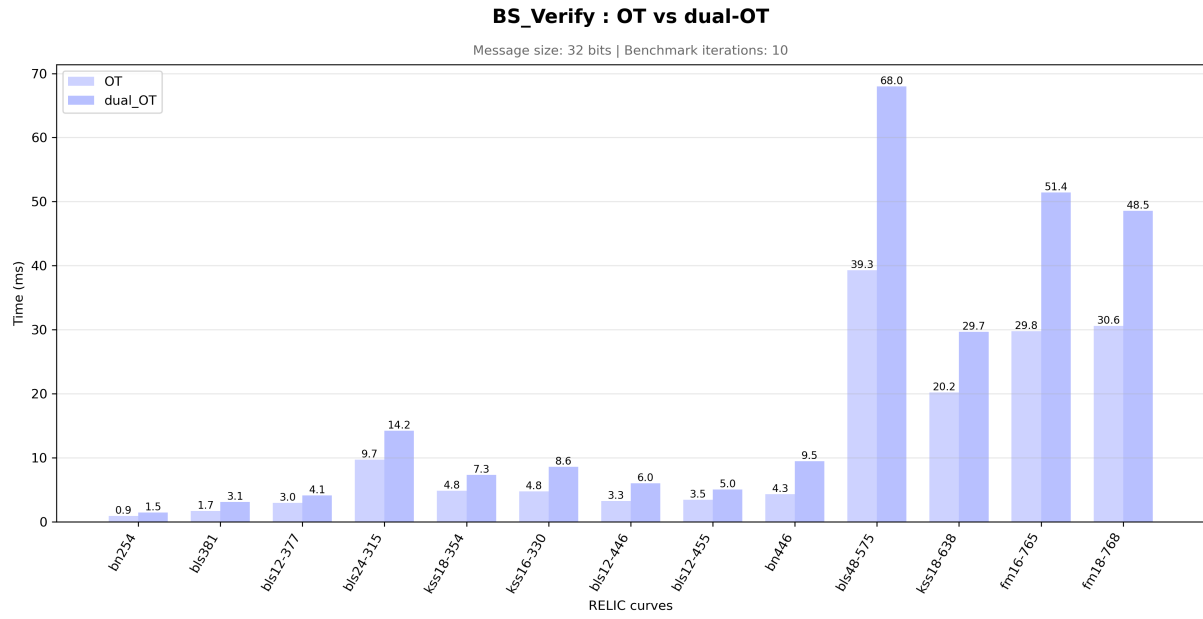


FIGURE 8 – Temps d'exécution de la vérification pour les protocoles OT et dual-OT.